Kotlin Collections — 120 Practice Questions (Question + Sample Input / Output)

A. Lists — Core Operations (1–12)

1. 1. Create an immutable list and a mutable list; append/remove safely.

Input: Input: Create an immutable list of [1,2,3] and a mutable list [10,20]. Operation: Add 30 to mutable, try to add
to immutable (should fail). Output: Output: Immutable remains [1,2,3]. Mutable after add -> [10,20,30].

2. 2. Convert an IntRange to a List and vice-versa.

Input: Input: Range 1..5 and List [1,2,3,4,5]. Output: Output: Range 1..5 -> List [1,2,3,4,5]. List -> Range not exact but can be represented as 1..5.

3. 3. Remove duplicates while preserving order.

Input: Input: [1, 2, 3, 2, 4, 1, 5] Output: Output: [1, 2, 3, 4, 5]

4. 4. Keep only consecutive duplicates removed (dedupe runs).

Input: Input: [1,1,2,2,2,3,1,1] Output: Output: [1,2,3,1]

5. 5. Take first N / last N elements; drop first N / last N.

Input: Input: [10,20,30,40,50], take first 2, drop last 1 Output: Output: take -> [10,20], drop last -> [10,20,30,40]

6. 6. Take / drop while a predicate holds (takeWhile/dropWhile).

Input: Input: [1,2,3,0,4], takeWhile { it>0 } Output: Output: [1,2,3]

7. 7. Get element at index with default if out of bounds (getOrNull/getOrElse).

Input: Input: list [5,6,7], request index 5 with default -1 Output: Output: getOrNull -> null, getOrElse -> -1

8. 8. Replace element at index in a new list (immutable update).

Input: Input: [a,b,c], replace index 1 with 'x' immutably Output: Output: New list -> [a,x,c], original -> [a,b,c]

9. 9. Split list at an index into two lists.

Input: Input: [1,2,3,4,5], split at index 2 Output: Output: [1,2] and [3,4,5]

10. 10. Rotate a list by k positions (left/right).

Input: Input: [1,2,3,4,5], rotate right by 2 Output: Output: [4,5,1,2,3]

11. 11. Reverse a subrange in a list (in-place and immutable).

Input: Input: [1,2,3,4,5], reverse subrange 1..3 Output: Output: In-place -> [1,4,3,2,5], immutable -> new list [1,4,3,2,5]

12. 12. Find indices of all occurrences of a given value.

Input: Input: [2,3,2,4,2], value=2 Output: Output: [0,2,4]

B. Filtering & Searching (13–24)

13. 13. Filter even numbers; separate evens/odds using partition.

Input: Input: [1,2,3,4,5,6] Output: Output: Evens=[2,4,6], Odds=[1,3,5]

14. 14. Remove nulls from List (filterNotNull).

Input: Input: [1,null,2,null,3] Output: Output: [1,2,3]

15. 15. Keep only non-blank strings; trim all items.

Input: Input: [" apple ", " ", "banana", ""] Output: Output: ["apple", "banana"]

16. 16. Find first/last element matching predicate; handle empty.

Input: Input: [1,4,6,7], first >5, last even Output: Output: first >5 -> 6, last even -> 6

17. 17. Find index of first/last match (indexOfFirst/indexOfLast).

Input: Input: ["a","b","c","b"], find 'b' Output: Output: indexOfFirst=1, indexOfLast=3

18. 18. Check any/all/none for a condition.

Input: Input: [2,4,6], condition is all elements even Output: Output: any -> true, all -> true, none -> false

19. 19. Remove items in another collection (subtract).

Input: Input: [1,2,3,4], remove [2,4] Output: Output: [1,3]

20. 20. Exclude items by predicate (filterNot).

Input: Input: [1,2,3,4,5], filterNot { it%2==0 } Output: Output: [1,3,5]

21. 21. Keep only unique items by key (distinctBy on data class field).

Input: Input: list of users [{id:1,email:a},{id:2,email:b},{id:3,email:a}] distinctBy email Output: Output: [{id:1,email:a},{id:2,email:b}] (first occurrence kept)

22. 22. Case-insensitive filtering on strings.

Input: Input: ["Apple","apple","Banana"], filter 'apple' ignore case Output: Output: ["Apple","apple"]

23. 23. Safely remove items while iterating (use iterator/remove()).

Input: Input: MutableList [1,2,3,4,5], remove even while iterating Output: Output: [1,3,5]

24. 24. Filter a deeply nested structure (list of lists) by inner predicate.

Input: Input: [[1,2],[3,4],[5,6]], keep inner lists having any even number Output: Output: [[1,2],[3,4],[5,6]] (all contain even)

C. Transforming & Mapping (25–36)

25. 25. Square numbers with map; produce strings with formatting.

Input: Input: [1,2,3], map to '1 -> 1', '2 -> 4' ... Output: Output: ["1 -> 1","2 -> 4","3 -> 9"]

26. 26. Use mapIndexed to include indices in results.

Input: Input: ["a","b","c"] Output: Output: ["0:a","1:b","2:c"]

27. 27. Map to pairs and build a map (associate/associateBy).

Input: Input: ["a","bb","ccc"], associateBy length Output: Output: {1=a, 2=bb, 3=ccc}

28. 28. Convert snake_case to camelCase for all strings.

Input: Input: ["first_name","last_name"] Output: Output: ["firstName","lastName"]

29. 29. flatMap: explode each item into zero/many items.

Input: Input: ["hi","ok"], map each to chars Output: Output: ["h","i","o","k"]

30. 30. Flatten a list of lists (flatten) vs flatMap { it }.

Input: Input: [[1,2],[3],[4,5]] Output: Output: [1,2,3,4,5]

31. 31. Zip two lists into pairs; unzip back to lists.

Input: Input: [1,2,3], ["a","b","c"] Output: Output: zipped=[(1,a),(2,b),(3,c)], unzipped -> [1,2,3] & [a,b,c]

32. 32. Zip lists of different lengths with zip + padding.

Input: Input: [1,2], ["a","b","c"], pad missing with null Output: Output: [(1,a),(2,b),(null,c)] or zippedUsing indices -> [(1,a),(2,b)] depending method

33. 33. Transpose a matrix using zip on lists of lists.

Input: Input: [[1,2,3],[4,5,6]] Output: Output: [[1,4],[2,5],[3,6]]

34. 34. Build transformed list efficiently using buildList.

Input: Input: generate squares 1..5 using buildList Output: Output: [1,4,9,16,25]

35. 35. Replace elements conditionally (map vs mapNotNull).

Input: Input: [1,2,3], replace even with null and drop null Output: Output: [1,3]

36. 36. Compute cartesian product of two lists.

Input: Input: [1,2] x ["a","b"] Output: Output: [(1,a),(1,b),(2,a),(2,b)]

D. Aggregations & Reductions (37–48)

37. 37. Sum/average/min/max of numbers.

Input: Input: [5,10,15] Output: Output: sum=30, average=10.0, min=5, max=15

38. 38. Custom aggregation with reduce vs fold.

Input: Input: [1,2,3,4], reduce to string '1-2-3-4' Output: Output: '1-2-3-4'

39. 39. Safe reduce for empty lists (reduceOrNull or fold with identity).

Input: Input: [], reduceOrNull sum Output: Output: null (or 0 with fold(0))

40. 40. Running totals: runningReduce and runningFold.

Input: Input: [1,2,3,4] Output: Output: running totals [1,3,6,10]

41. 41. Prefix sums of a list of numbers.

Input: Input: [1,1,1,1], prefix sums Output: Output: [1,2,3,4]

42. 42. Product of numbers with overflow awareness (Long).

Input: Input: [2,3,5], product Output: Output: 30

43. 43. Median of a list (odd/even count).

Input: Input: [5,1,9,2,8] Output: Output: median=5

44. 44. Mode (most frequent element) with tie-breaker.

Input: Input: [1,2,2,3,3] Output: Output: mode could be 2 or 3 (tie) — choose lowest -> 2

45. 45. Kth smallest/largest without sorting whole list (selection algos).

Input: Input: [7,2,5,3,9], k=2 largest Output: Output: 7 (2nd largest)

46. 46. Count elements matching predicate.

Input: Input: [1,2,3,4,5], count even Output: Output: 2

47. 47. Weighted average from (value, weight) pairs.

Input: Input: [(70,0.3),(90,0.7)] Output: Output: weighted average = 84.0

48. 48. Binary to decimal from a list of bits with fold.

Input: Input: [1,0,1,1] (MSB->LSB) Output: Output: 11 (1*8 +0*4 +1*2 +1*1)

E. Grouping & Counting (49–60)

49. 49. Word frequency map from a paragraph (normalize case/punctuation).

Input: Input: 'Hello, hello world! Hello.' Output: Output: {hello=3, world=1}

50. 50. Group strings by first letter (groupBy).

Input: Input: ["apple","banana","avocado","berry"] Output: Output: {a=[apple,avocado], b=[banana,berry]}

51. 51. Group people by age range buckets (0-9,10-19...).

Input: Input: ages [5,12,27,34,42] Output: Output: {0s=[5],10s=[12],20s=[27],30s=[34],40s=[42]}

52. 52. Group and transform values (groupBy + mapValues).

Input: Input: ['aa','ab','bb','bc'] grouped by first char and lengths Output: Output: {a=[2,2], b=[2,2]}

53. 53. groupingBy().eachCount() vs groupBy().mapValues { it.size }.

Input: Input: ['x','y','x','z','x'] Output: Output: {x=3,y=1,z=1}

54. 54. Multi-level grouping (department -> team).

Input: Input: [(dept:Eng,team:A,name:Tom),(dept:Eng,team:B,name:Jen),(dept:HR,team:C,name:Sam)] Output: Output:
{Eng={A=[Tom],B=[Jen]}, HR={C=[Sam]}}

55. 55. Build histogram bins for numbers.

Input: Input: [1,2,2,3,7,8], bin size=3 (0-2,3-5,6-8) Output: Output: {0-2=[1,2,2], 3-5=[3], 6-8=[7,8]}

56. 56. Stable grouping preserving original order of groups.

Input: Input: ['b','a','b','c'] grouped by letter with first occurrence order Output: Output: groups in order b->a->c

57. 57. Find top-N frequent items from a large list.

Input: Input: [a,a,a,b,b,c], top2 Output: Output: [a,b]

58. 58. Invert a map to value -> list of keys (handle collisions).

Input: Input: {k1=1,k2=2,k3=1} Output: Output: {1=[k1,k3],2=[k2]}

59. 59. Group anagrams together (signature by sorted chars).

Input: Input: ["eat","tea","tan","ate","nat"] Output: Output: {aet=[eat,tea,ate], ant=[tan,nat]}

60. 60. Bucket emails by domain.

Input: Input: ["a@gmail.com","b@yahoo.com","c@gmail.com"] Output: Output: {gmail.com=[a,b], yahoo.com=[b]} (names
simplified)

F. Sorting & Ordering (61–72)

61. 61. Sort ascending/descending (sorted, sortedDescending).

Input: Input: [3,1,4,2] Output: Output: ascending [1,2,3,4], descending [4,3,2,1]

62. 62. Sort data class list by one property (sortedBy).

Input: Input: [{name:A,age:30},{name:B,age:25}] sortedBy age Output: Output: [{B,25},{A,30}]

63. 63. Sort by multiple keys (thenBy, thenByDescending).

Input: Input: people [{name:A,age:30,score:90},{name:B,age:30,score:95}] sort by age then score desc Output: Output:
[{B,30,95},{A,30,90}]

64. 64. Case-insensitive sort for strings with locale.

Input: Input: ["apple","Banana","apple"] Output: Output: ["apple","apple","Banana"] (case-insensitive order)

65. 65. Stable vs unstable sorts (show effect).

Input: Input: list of pairs [(1,'a'),(1,'b')] sorted by first only Output: Output: stable sort keeps order ->
[(1,'a'),(1,'b')]

66. 66. Randomize order (shuffled) with a deterministic seed.

Input: Input: [1,2,3,4], seed=42 Output: Output: deterministic shuffled e.g. [3,1,4,2] (example)

67. 67. Custom comparator with Comparator { a,b -> ... }.

Input: Input: ["a1","a10","a2"] natural-number-aware comparator Output: Output: ["a1","a2","a10"]

68. 68. Natural sort on strings with numbers (file2 before file10) — approach.

Input: Input: ["file10","file2","file1"] Output: Output: ["file1","file2","file10"]

69. 69. Sort only a subrange of a mutable list.

Input: Input: [5,4,3,2,1], sort subrange 1..3 Output: Output: [5,2,3,4,1]

70. 70. Top-K largest by property without full sort (partial selection).

Input: Input: [5,1,8,7,3], k=2 largest Output: Output: [8,7]

71. 71. Sort map by keys; sort map by values returning LinkedHashMap.

Input: Input: {b=2,a=1,c=3} Output: Output: sortedByKey -> {a=1,b=2,c=3}, sortedByValueDesc -> {c=3,b=2,a=1}

72. 72. Keep relative order while removing duplicates (distinct keeps first).

Input: Input: [2,1,2,3,1] Output: Output: [2,1,3]

G. Sets & Set-like Tasks (73–84)

73. 73. Union, intersection, difference of sets.

Input: Input: A={1,2,3}, B={2,3,4} Output: Output: union={1,2,3,4}, intersection={2,3}, difference A-B={1}

74. 74. Symmetric difference (elements in either set, but not both).

Input: Input: A={1,2,3}, B={2,3,4} Output: Output: {1,4}

75. 75. Test subset/superset/disjoint.

Input: Input: A={1,2}, B={1,2,3}, C={4,5} Output: Output: A subset of B=true, A disjoint with C=true

76. 76. Keep insertion order using linkedSetOf (LinkedHashSet).

Input: Input: insert in order [b,a,c,b] into linked set Output: Output: [b,a,c] (insertion order)

77. 77. Case-insensitive set of strings (normalize or custom wrapper).

Input: Input: ["A","a","B"] Output: Output: ["A","B"] (case-insensitive unique)

78. 78. Build a set of unique objects by key with distinctBy vs toSet.

Input: Input: users [{id:1,email:a},{id:2,email:a},{id:3,email:b}] distinctBy email Output: Output: [{id:1,email:a},{id:3,email:b}]

79. 79. Compute Jaccard similarity of two sets.

Input: Input: A={1,2,3}, B={2,3,4} Output: Output: intersection=2, union=4, Jaccard=0.5

80. 80. Convert list to set and back without losing order.

Input: Input: [2,1,2,3,1] Output: Output: to LinkedHashSet -> [2,1,3], back to list -> [2,1,3]

81. 81. Remove from set while iterating (use iterator.remove()).

Input: Input: set {1,2,3,4}, remove evens while iterating Output: Output: {1,3}

82. 82. Fast membership checks vs list (in with set).

Input: Input: large collection membership test of 10000 items, test element x Output: Output: set contains -> O(1) faster than list O(n) (true/false)

83. 83. Build an ordered unique list (LinkedHashSet then toList).

Input: Input: ["a","b","a","c"] Output: Output: ["a","b","c"]

84. 84. Intersect many sets efficiently (reduce/intersect).

Input: Input: [{1,2,3},{2,3,4},{2,3}] Output: Output: intersection -> {2,3}

H. Maps — Creation, Query, Update (85–99)

85. 85. Create immutable and mutable maps; read safely with default (getOrDefault).

Input: Input: mapOf(a=1,b=2), get 'c' with default 0 Output: Output: getOrDefault('c',0) -> 0

86. 86. Update or insert with getOrPut.

Input: Input: mutableMap = {}, getOrPut 'x' with default list and add value 1 Output: Output: {'x'=[1]}

87. 87. Merge two maps with conflict resolution function.

Input: Input: m1={a=1,b=2}, m2={b=3,c=4}, merge sums on conflict Output: Output: {a=1,b=5,c=4}

88. 88. Count frequency with MutableMap.compute style (getOrPut + increment).

Input: Input: ['a','b','a','c'] Output: Output: {a=2,b=1,c=1}

89. 89. Filter entries by key and/or value (filterKeys/filterValues).

Input: Input: {a=10,b=5,c=20}, filter values >=10 Output: Output: {a=10,c=20}

90. 90. Transform keys/values (mapKeys/mapValues) and note laziness.

Input: Input: {a=1,b=2}, mapKeys to uppercase Output: Output: {A=1,B=2}

91. 91. Sort map by keys; keep order in result.

Input: Input: {b=2,a=1,c=3} Output: Output: {a=1,b=2,c=3} (LinkedHashMap)

92. 92. Sort map by values; keep top-N pairs.

Input: Input: {a=5,b=2,c=9}, top2 by value Output: Output: {c=9,a=5}

93. 93. Reverse a 1-1 map (key<->value); handle duplicates defensively.

Input: Input: {k1=1,k2=2,k3=1} Output: Output: cannot invert to one-to-one for value 1 unless grouping -> {1=[k1,k3],2=[k2]}

94. 94. Build index map: id -> object using associateBy.

Input: Input: list users [{id:10,name:A},{id:11,name:B}] Output: Output: {10={...},11={...}}

95. 95. Multi-map: key -> List using groupBy vs manual getOrPut.

Input: Input: pairs [(a,1),(b,2),(a,3)] Output: Output: {a=[1,3], b=[2]}

96. 96. Immutable copy after updates (toMap) to expose read-only view.

Input: Input: mutableMap updated then toMap() returned Output: Output: callers see immutable snapshot

97. 97. Safe iteration while modifying: collect keys to change first.

Input: Input: map {a=1,b=2}, remove where value<2 during iteration Output: Output: {b=2} after removals

98. 98. LRU-like cache with access-order LinkedHashMap (Java interop).

Input: Input: insert keys [a,b,c], access 'a', insert 'd' with maxSize=3 Output: Output: evict least-recently-used -> maybe 'b' removed, cache contains {c,a,d} depending access

99. 99. Case-insensitive map strategies (normalize keys or custom wrapper).

Input: Input: put('Key',1), get('key') Output: Output: returns 1 if normalized to lowercase keys

I. Sequences & Performance (100–108)

100. 100. Convert a large list pipeline to asSequence(); measure impact (lazy evaluation).

Input: Input: large list 1..1000000 then map/filter/first Output: Output: asSequence finds first match without materializing full intermediate lists (faster, lower memory)

101. 101. Demonstrate intermediate vs terminal operations; avoid materialization.

Input: Input: list.map{...}.filter{...}.toList() vs asSequence().map{...}.filter{...}.toList() Output: Output: sequence avoids creating intermediate lists

102. 102. Sequence with generateSequence (e.g., Collatz) and takeWhile.

Input: Input: start 13 collatz sequence takeWhile >1 Output: Output: [13,40,20,10,5,16,8,4,2,1]

103. 103. Lazy file line processing with sequences.

Input: Input: large file lines, find first line containing 'ERROR' Output: Output: returns first matching line without loading whole file

104. 104. Compare map+filter+toList vs asSequence().map…filter…toList().

Input: Input: small list vs huge list Output: Output: sequences help only for huge lists/pipelines

105. 105. Break early with sequences (e.g., find first heavy match).

Input: Input: list of expensive computations, find first even result Output: Output: stops after finding first match

106. 106. Avoid flatMap explosion with sequences for huge data.

Input: Input: huge nested lists with flatMap then filter Output: Output: use sequence to avoid generating huge intermediate collection

107. 107. When not to use sequences (tiny lists / random access needed).

Input: Input: small list [1,2,3] many random accesses expected Output: Output: prefer lists over sequences

108. 108. Parallelism caveat: sequences aren't parallel—discuss alternatives.

Input: Input: CPU-bound processing of large list Output: Output: consider coroutines/parallel streams/Java ForkJoin for
parallelism

J. Windows, Chunking, Sliding (109–114)

109. 109. Split list into equal chunks (chunked); handle last short chunk.

Input: Input: [1,2,3,4,5,6,7], chunk size 3 Output: Output: [[1,2,3],[4,5,6],[7]]

110. 110. Fixed-size sliding window with windowed(size, step, partialWindows).

Input: Input: [1,2,3,4,5], windowed size=3 step=1 Output: Output: [[1,2,3],[2,3,4],[3,4,5]]

111. 111. Compute moving average with windowed.

Input: Input: [1,2,3,4,5], window size 3, compute average Output: Output: [2.0,3.0,4.0]

112. 112. Pair each element with its next/previous (adjacent pairs).

Input: Input: [10,20,30,40], adjacent pairs Output: Output: [(10,20),(20,30),(30,40)]

113. 113. Find increasing subarray runs using windowed(2).

Input: Input: [1,2,2,3,4,1], increasing runs Output: Output: runs e.g. [1,2],[2,3],[3,4]

114. 114. Page through a list (page index + page size to subList bounds).

Input: Input: list 1..50, pageIndex=2 (0-based), pageSize=10 Output: Output: items [21..30]

K. Real-World Patterns (115–120)

115. 115. Merge two sorted lists into one sorted list (linear time).

Input: Input: [1,3,5], [2,4,6] (both sorted) Output: Output: [1,2,3,4,5,6]

116. 116. Merge shopping carts by productId, summing quantities.

Input: Input: cart1 [(p1,2),(p2,1)], cart2 [(p1,1),(p3,4)] Output: Output: merged [(p1,3),(p2,1),(p3,4)]

117. 117. Deduplicate users by email, keeping the last occurrence.

Input: Input: [{id:1,email:a},{id:2,email:b},{id:3,email:a}] keep last Output: Output: [{id:2,email:b},{id:3,email:a}] (order may be preserved as last seen)

118. 118. Case-insensitive unique usernames while preserving original casing.

Input: Input: ["Alice","alice","ALICE","Bob"] Output: Output: ["ALICE","Bob"] or keep first occurrence depending rule

(example keep last-> ["ALICE","Bob"])

119. 119. From two lists ids and items, keep items whose id exists (associateBy + filter).

Input: Input: ids [2,3], items [{id:1,v:...},{id:2,v:...},{id:3,v:...}] Output: Output: items with id 2 and 3

120. 120. Build a name directory: initial -> sorted list of names with locale rules.

Input: Input: ["alice","Álvaro","bob","Anna"] Output: Output: {A=[alice,Álvaro,Anna], B=[bob]} (locale-aware sort example)